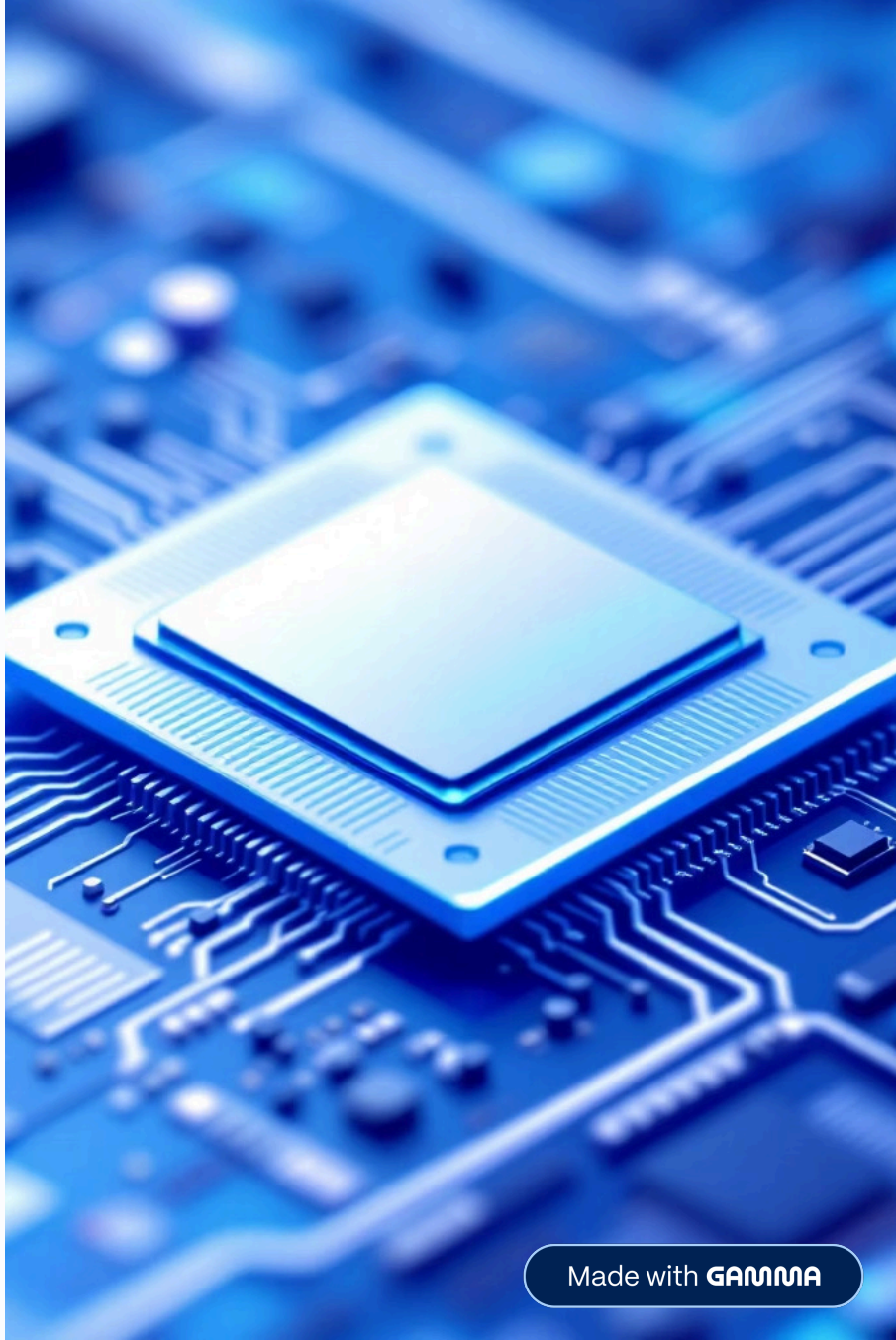
Sistemas Operacionais e Operações Lógicas e Aritméticas

Aula 4: Fundamentos de Sistemas Operacionais e Arquitetura de Computadores

By Prof. Cloves Rocha



Made with GAMMA

Definição de Sistema Operacional

Um **Sistema Operacional (SO)** é o software mais fundamental de um computador, funcionando como intermediário entre o hardware e os programas aplicativos. Ele gerencia os recursos do sistema, controla a execução de processos e fornece uma interface para que os usuários possam interagir com a máquina.

Gerenciamento de Recursos

Controla e aloca recursos do sistema como CPU, memória, dispositivos de entrada/saída e armazenamento

Proteção e Segurança

Implementa mecanismos de segurança para proteger dados e controlar acesso aos recursos do sistema

Interface de Abstração

Fornece uma camada de abstração que simplifica a interação com o hardware complexo

Os sistemas operacionais modernos permitem a execução de múltiplos programas simultaneamente, gerenciam a memória de forma eficiente, implementam sistemas de arquivos para organização de dados e fornecem serviços de rede para comunicação entre computadores.

Made with GAMMA

Distribuições Linux

O Linux é um dos sistemas operacionais mais versáteis e customizáveis disponíveis hoje. Sua arquitetura de código aberto permitiu o desenvolvimento de centenas de distribuições, cada uma otimizada para propósitos específicos.

1. **Ubuntu**
   Distribuição amigável para iniciantes, ideal para desktops e servidores. Base para muitas outras distros
2. **Debian**
   Sistema estável e confiável, focado em software livre. Base para Ubuntu e muitas outras distribuições
3. **CentOS/RHEL**
   Distribuições empresariais focadas em servidores. Oferecem suporte de longo prazo e estabilidade
4. **Fedora**
   Plataforma de ponta com as últimas tecnologias. Testa novos recursos antes de chegarem ao RHEL
5. **Arch Linux**
   Distribuição rolling-release para usuários avançados. Máximo controle e customização do sistema

Servidores

CentOS, RHEL, Ubuntu Server - otimizados para estabilidade e desempenho em ambientes de produção

Desktop

Ubuntu, Linux Mint, Fedora - interfaces amigáveis e suporte para aplicações de usuário final

Embarcados

OpenWrt, Yocto - sistemas compactos para roteadores, IoT e dispositivos com recursos limitados

Made with GAMMA

Sistema de Avaliação e Projetos

Critérios de Avaliação

O sistema de avaliação desta disciplina combina avaliação teórica com prática, permitindo aos estudantes demonstrar compreensão conceitual e habilidades técnicas aplicadas.

- **Avaliações teóricas (40%)**: Provas e questionários sobre conceitos fundamentais de arquitetura de computadores
- **Projetos práticos (35%)**: Implementação de circuitos lógicos e operações aritméticas usando ferramentas de simulação
- **Participação e atividades (15%)**: Discussões em aula, resolução de exercícios e contribuições para o aprendizado coletivo
- **Apresentação do projeto (10%)**: Comunicação clara dos resultados e metodologia utilizada

Projetos Disponíveis

Os estudantes podem escolher entre diferentes projetos que aplicam os conceitos aprendidos em situações práticas do mundo real.

- **Projeto 1**: Desenvolvimento de um simulador de operações aritméticas e lógicas
- **Projeto 2**: Implementação de circuitos combinacionais usando portas lógicas básicas
- **Projeto 3**: Construção de uma Unidade Lógica Aritmética (ULA) simplificada
- **Projeto 4**: Análise comparativa de performance de diferentes arquiteturas de processadores

Formação de Equipes: Os projetos devem ser realizados em equipes de 3 a 6 estudantes. Cada equipe deverá apresentar seus resultados em sessões de demonstração ao longo do semestre. A colaboração e divisão de tarefas serão considerados na avaliação final.

Made with GAMMA

Operações Lógicas e Aritméticas: Fundamentos

Números com Sinal

Representam valores positivos e negativos usando o bit mais significativo como indicador de sinal (0 para positivo, 1 para negativo)

Números sem Sinal

Representam apenas valores não-negativos, utilizando todos os bits para magnitude do número

As operações lógicas e aritméticas formam a base de toda computação digital. Toda operação executada por um processador, desde cálculos matemáticos simples até execução de programas complexos, se reduz a operações fundamentais em bits.

Representação de Números Binários

Em sistemas digitais, os números são representados em **base 2**, usando apenas os dígitos 0 e 1. A compreensão de como representar e manipular números binários é fundamental para entender operações aritméticas e lógicas.

Representação sem Sinal

Para um número de **n bits**, podemos representar valores de 0 até 2^n - 1. Por exemplo, com 8 bits, representamos valores de 0 a 255.

Exemplo (8 bits): 10010110 = 1×128 + 0×64 + 0×32 + 1×16 + 0×8 + 1×4 + 1×2 + 0×1 = 150

Representação com Sinal

Usa o **bit de sinal** (MSB - Most Significant Bit) para indicar positivo (0) ou negativo (1). Valores de -2^(n-1) até 2^(n-1)-1.

Exemplo (8 bits): 10010110 = -1×128 + 0×64 + 0×32 + 1×16 + 0×8 + 1×4 + 1×2 + 0×1 = -110

Aritmética Binária

01

Adição Binária

Segue regras similares à adição decimal, com carry quando 1+1=10

02

Subtração Binária

Implementada usando complemento de 2 para representar números negativos

03

Overflow

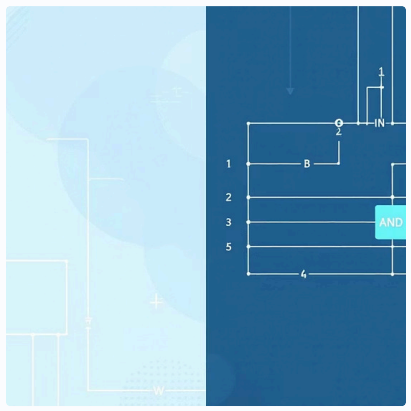
Ocorre quando o resultado não cabe no número de bits disponíveis

Importante: O processador não "sabe" se estamos usando números com sinal ou sem sinal - cabe ao programador interpretar corretamente os resultados com base na representação escolhida.

Made with GAMMA

Principais Portas Lógicas

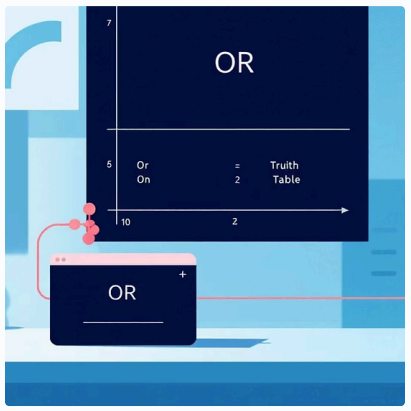
As **portas lógicas** são os blocos fundamentais de construção de circuitos digitais. Cada porta implementa uma operação booleana básica, e combinando essas portas podemos criar circuitos de complexidade arbitrária.



Porta AND

Retorna 1 apenas quando todas as entradas são 1. Operação lógica de conjunção.

Exemplo: A AND B = 1 se A=1 e B=1



Porta OR

Retorna 1 quando pelo menos uma entrada é 1. Operação lógica de disjunção.

Exemplo: A OR B = 1 se A=1 ou B=1

Porta NOT

Inverte o valor de entrada. Operação lógica de negação ou complemento.

Exemplo: NOT A = 1 se A=0

Porta NAND

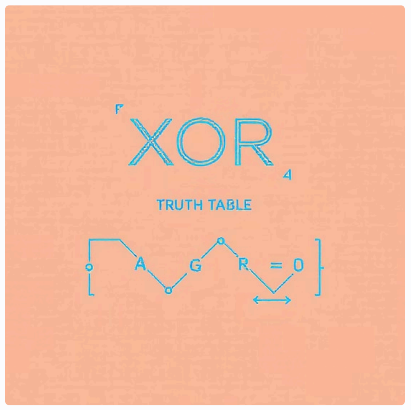
AND seguido de NOT. Retorna 0 apenas quando todas as entradas são 1.

Turing-complete: Pode implementar qualquer função booleana

Porta NOR

OR seguido de NOT. Retorna 1 apenas quando todas as entradas são 0.

Turing-complete: Também pode implementar qualquer função



Porta XOR

OR exclusivo. Retorna 1 quando entradas são diferentes.

Útil: Detecção de diferenças e operações de paridade

Tabela Verdade Completa

A	B	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Made with GAMMA

Adição e Subtração Binária

Adição Binária

A adição binária segue regras simples mas fundamentais. Quando somamos 1 + 1, precisamos de um **carry bit** (vai-um) para representar o resultado, pois 1 + 1 = 10 em binário.

1

Bit a Bit

Processamos cada posição binária individualmente, da direita para esquerda

2

Carry Propagação

O carry pode propagar através de múltiplos bits durante a adição

3

Resultado Final

Obtemos soma e carry final que indica overflow se necessário

Exemplo de Adição

Operação: 1011 + 0110

Resultado: 10001

Processo de cálculo:

```
 1011  (11)
+ 0110  (6)
------
 10001  (17)
```

Carry propagou do bit 0 para bit 1, depois para bit 2, gerando um bit extra (overflow em 4 bits)

Half Adder (Somador Meio)

Implementa soma de dois bits, gerando soma e carry

- Entradas: A, B
- Saídas: Soma (A XOR B), Carry (A AND B)

Full Adder (Somador Completo)

Implementa soma com carry de entrada

- Entradas: A, B, Carry-in
- Saída: Soma, Carry-out

Subtração Binária usando Complemento de 2

A subtração é implementada como **A - B = A + (-B)**, onde -B é obtido usando complemento de 2.

Passo 1: Inverter Bits

Aplicar NOT em todos os bits de B

Passo 2: Adicionar 1

Adicionar 1 ao resultado do passo 1

Passo 3: Somar

A + complemento de 2 de B

Overflow: Ocorre quando o resultado não cabe no formato. Em complemento de 2, detectamos overflow quando há carry inconsistente entre bits ou quando sinais dos operandos são iguais mas sinal do resultado é diferente.

Made with GAMMA

Operações Lógicas

As **operações lógicas** manipulam bits individualmente, aplicando funções booleanas. Diferente das operações aritméticas, não consideram valores numéricos mas apenas estados lógicos (0 ou 1).

AND Bitwise

Operação E lógica bit a bit. Útil para mascaramento de bits.

Exemplo: 1010 AND 1100 = 1000

OR Bitwise

Operação OU lógica bit a bit. Útil para forçar bits a 1.

Exemplo: 1010 OR 1100 = 1110

XOR Bitwise

Operação OU exclusivo bit a bit. Útil para alternar bits e criptografia simples.

Exemplo: 1010 XOR 1100 = 0110

NOT Bitwise

Negação lógica de todos os bits. Complemento de 1.

Exemplo: NOT 1010 = 0101

Operações de Deslocamento (Shift)

Left Shift (<<)

Desloca bits para esquerda, preenchendo com zeros à direita

Equivalente: Multiplicação por 2^n

Exemplo: 0010 << 2 = 1000 (2 × 4 = 8)

- Overflow bits são descartados
- Usado para multiplicação rápida

Right Shift (>>)

Desloca bits para direita, preenchendo com bit de sinal (aritmético) ou zero (lógico)

Equivalente: Divisão por 2^n

Exemplo: 1000 >> 2 = 0010 (8 ÷ 4 = 2)

- Bits descartados são perdidos
- Usado para divisão rápida

Aplicações Práticas

Mascaramento de Bits

Usando AND com máscara para extrair ou limpar bits específicos

Exemplo: 10101100 AND 00001111 = 00001100 (mantém últimos 4 bits)

Extração de Campos

Combinando shift e AND para extrair campos de dados compactados

Exemplo: (10101100 >> 4) AND 00001111 = 1010 (extrai bits 4-7)

Compactação de Dados

Armazenando múltiplos valores em um único inteiro usando operações bitwise

Uso: Manipulação de pixels, flags de status, protocolos de rede

Dica de Programação: Operações bitwise são extremamente rápidas e são frequentemente usadas em sistemas embarcados, drivers de dispositivo e otimização de código de baixo nível.

Made with GAMMA

Unidade Lógica Aritmética (ULA)

A **Unidade Lógica Aritmética (ULA)** ou **Arithmetic Logic Unit (ALU)** é o coração de qualquer processador. É o componente responsável por executar todas as operações aritméticas e lógicas do computador.

Recebe Operandos

Dois valores de entrada (A e B) e código de operação

Executa Operação

Aplica operação especificada usando circuitos combinacionais

Produz Resultado

Resultado da operação e flags de status (zero, carry, overflow)

Componentes da ULA

Unidade Aritmética

Implementa operações como adição, subtração, incremento, decremento

Unidade Lógica

Implementa operações bitwise: AND, OR, XOR, NOT, shifts

Circuitos de Controle

Seleciona qual operação executar baseado no código de operação

Flags de Status

Indicadores de condição: zero, carry, overflow, sinal

Organização Interna

Entradas da ULA

- **Operandos A e B:** Valores a serem operados (tipicamente 32 ou 64 bits)
- **Código de Operação (Opcode):** Especifica qual operação executar
- **Carry-in:** Bit de carry de operações anteriores
- **Clock:** Sincronização das operações

Saídas da ULA

- **Resultado:** Saída da operação (mesmo tamanho dos operandos)
- **Flags:** Sinalizador de condições especiais
- **Carry-out:** Bit de carry gerado
- **Overflow:** Indica resultado fora do range

Flags de Status

Zero Flag

Setado quando resultado é zero. Útil para comparações e loops

Carry Flag

Setado quando há carry-out. Útil para operações com múltiplos precisões

Overflow Flag

Setado quando resultado não cabe em formato com sinal

Sign Flag

Copia bit de sinal do resultado. Indica se resultado é negativo

Arquitetura Moderna: ULA modernas são profundamente pipelined e podem executar múltiplas operações simultaneamente, incluindo operações de ponto flutuante em unidades separadas (FPU - Floating Point Unit).

Made with GAMMA

Desafio 3: Projeto Prático ULA

Objetivo do Desafio

Implementar uma **Unidade Lógica Aritmética (ULA)** simplificada que execute operações básicas de adição, subtração, AND, OR e XOR. O projeto deve ser desenvolvido em equipe de 3 a 6 estudantes e inclui implementação, testes e apresentação.

01

Design da Arquitetura

Definir entradas, saídas e operações suportadas. Criar diagrama de blocos da ULA

02

Implementação

Codificar ULA usando linguagem de descrição de hardware (VHDL/Verilog) ou ferramentas de simulação

03

Testes e Validação

Desenvolver testbench para validar todas as operações com casos de teste abrangentes

04

Documentação

Preparar relatório técnico explicando design, implementação e resultados dos testes

05

Apresentação

Demonstrar funcionamento da ULA em sessão de apresentação com toda equipe

Requisitos Técnicos

Funcionalidades Obrigatórias

1. Operações aritméticas: adição e subtração
2. Operações lógicas: AND, OR, XOR, NOT
3. Operações de shift: left shift e right shift
4. Implementação de flags de status (zero, carry, overflow, sinal)
5. Interface clara para seleção de operação

Critérios de Avaliação

1. **Corretude:** Funcionamento correto de todas operações
2. **Completude:** Implementação de requisitos obrigatórios
3. **Design:** Organização modular e clara do código
4. **Testes:** Cobertura completa e validação rigorosa
5. **Apresentação:** Clareza na comunicação dos resultados

4-8

Semanas de Desenvolvimento

Tempo previsto para conclusão do projeto

3-6

Estudantes por Equipe

Tamanho ideal para colaboração efetiva

100%

Cobertura de Testes

Meta para validação completa do sistema

Ferramentas Sugeridas

VHDL/Verilog

Linguagens de descrição de hardware para implementação de circuitos digitais

ModelSim/GTKWave

Ferramentas de simulação e visualização de formas de onda

Plataforma FPGA

Implementação física opcional em hardware reconfigurável

Entregáveis: Código fonte completo, relatório técnico com diagramas e análise, testbench com resultados, apresentação em formato de slides (10-15 minutos por equipe).

Made with GAMMA